

CSC 364: Processor Components Implementation

32-bit RISC-V ALU Design and Verification

1. Introduction

This report details the design and implementation of a 32-bit Arithmetic Logic Unit (ALU) for the RISC-V architecture. The ALU is designed to perform arithmetic, logical, shift, and comparison operations, integrated into a modular hardware system.

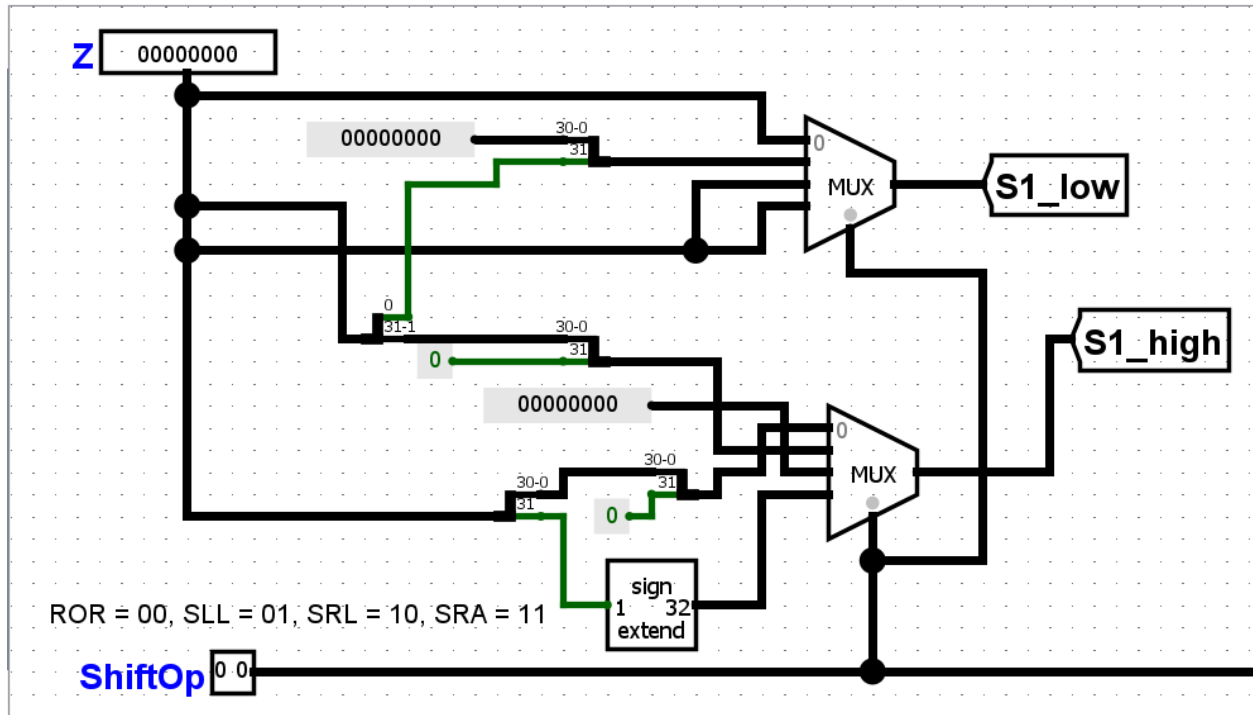
2. ALU Architecture & Design

The ALU follows a modular hierarchy, consisting of four primary functional units integrated by a 32-bit 4-to-1 Multiplexer.

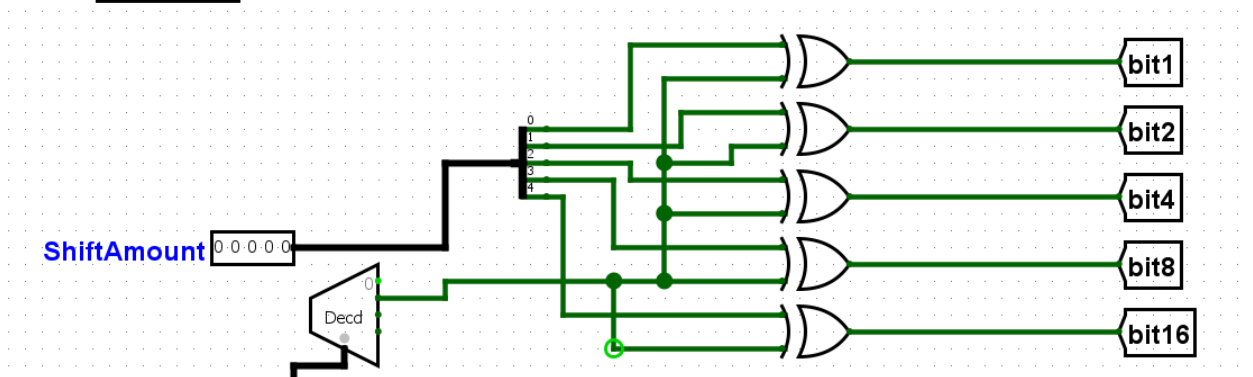
2.1 Shifter Unit

The Shifter Unit implements SLL, SRL, SRA, and ROR operations. While the lab manual suggested a single 63-bit extender, the provided version of Logisim only supports upto 32-bit architecture so I utilized a Low and High 32-bit paths to manage shifts.

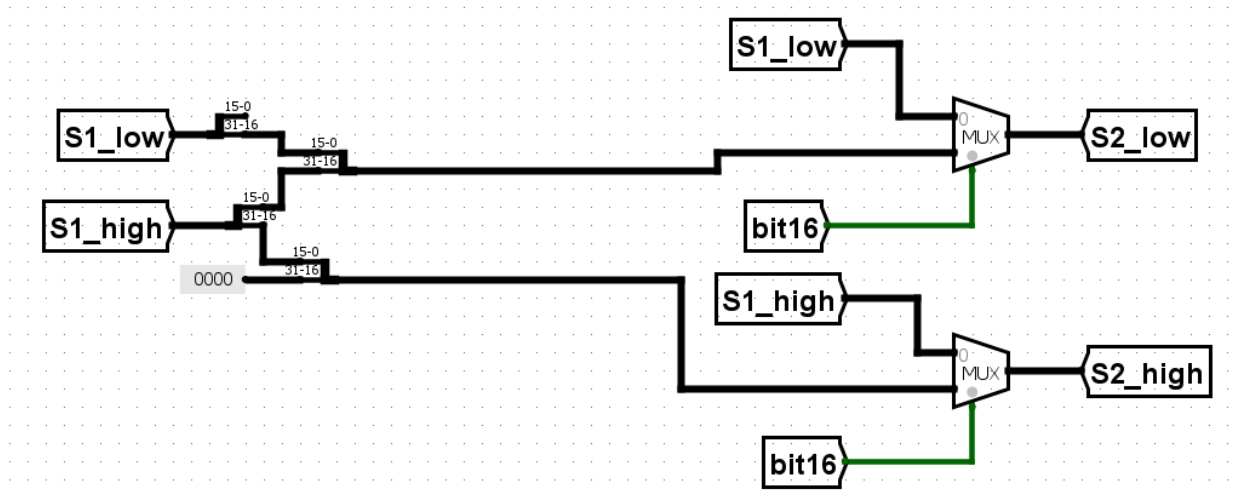
- **SRL (Shift Right Logical):** The Low bus is initialized with Z, and the High bus is initialized to 0. Right shifting moves data from Low toward the 0-filled High path.
- **SRA (Shift Right Arithmetic):** The Low bus is initialized with Z. The High bus is initialized with a 32-bit extension of the sign bit ($Z[31]$), ensuring the sign is preserved during right shifts.
- **ROR (Rotate Right):** Both the High and Low buses are initialized with Z. This effectively creates a 64-bit wrapping field where bits shifted out of the bottom of the Low bus are replaced by bits from the High bus.
- **SLL (Shift Left Logical):** To perform a left shift using right-shifting hardware, the design must address the "31-bit issue." Because the control logic uses a 1s-complement XOR ($31 - X$) rather than a 2s-complement ($32 - X$), a 1-bit offset is created. To correct this in hardware, the input data is pre-aligned during initialization: the Low bus is initialized with 31 zeros and the first bit of the input, while the High bus is initialized with a zero followed by the remaining 31 bits of the input data. This pre-shift allows the $31 - X$ right-shift to result in a logically correct left-shift of X.



- **Shift Amount Logic (SLL handling):** To perform Shift Left Logical (SLL) using the same right-shift hardware, an XOR-gate logic block is used on the ShiftAmount bits. This allows the shifter to interpret the left-shift as a mirrored right-shift operation.



- **Modular Shift Stages:** The unit consists of five stages (16, 8, 4, 2, 1). Each stage takes the High and Low inputs from the previous stage (e.g., S1_low, S1_high) and uses a 32-bit MUX to either pass the data or shift it by the corresponding power of two.

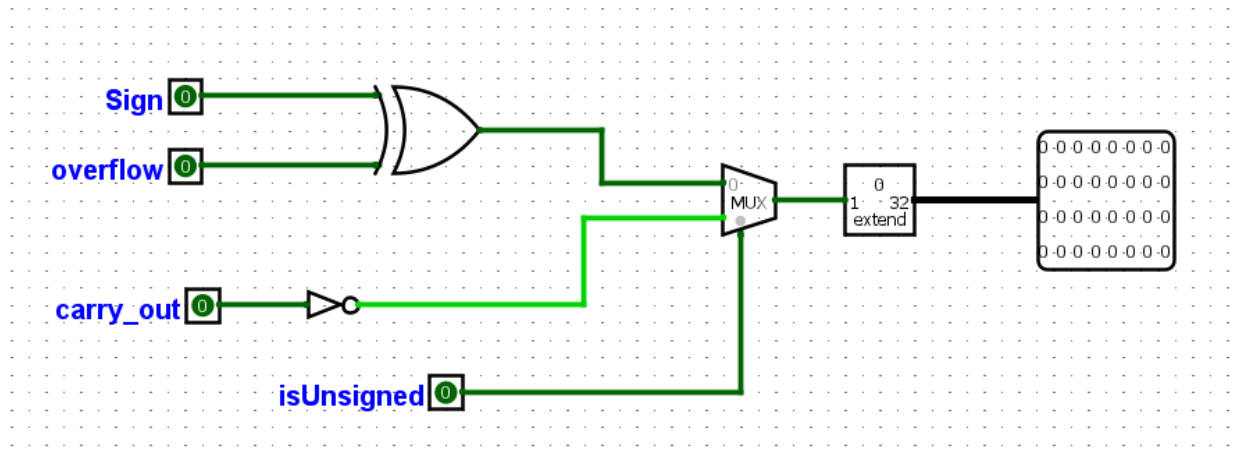


- **Result Selection:** At the final stage, only the "Low" 32-bit result is extracted as the final ALU output, ensuring the 32-bit RISC-V requirement is met.

2.2 Comparator Unit (SLT/SLTU)

The Comparator Unit determines if $A < B$ by analyzing the flags generated during the subtraction $A - B$ in the Arithmetic Unit.

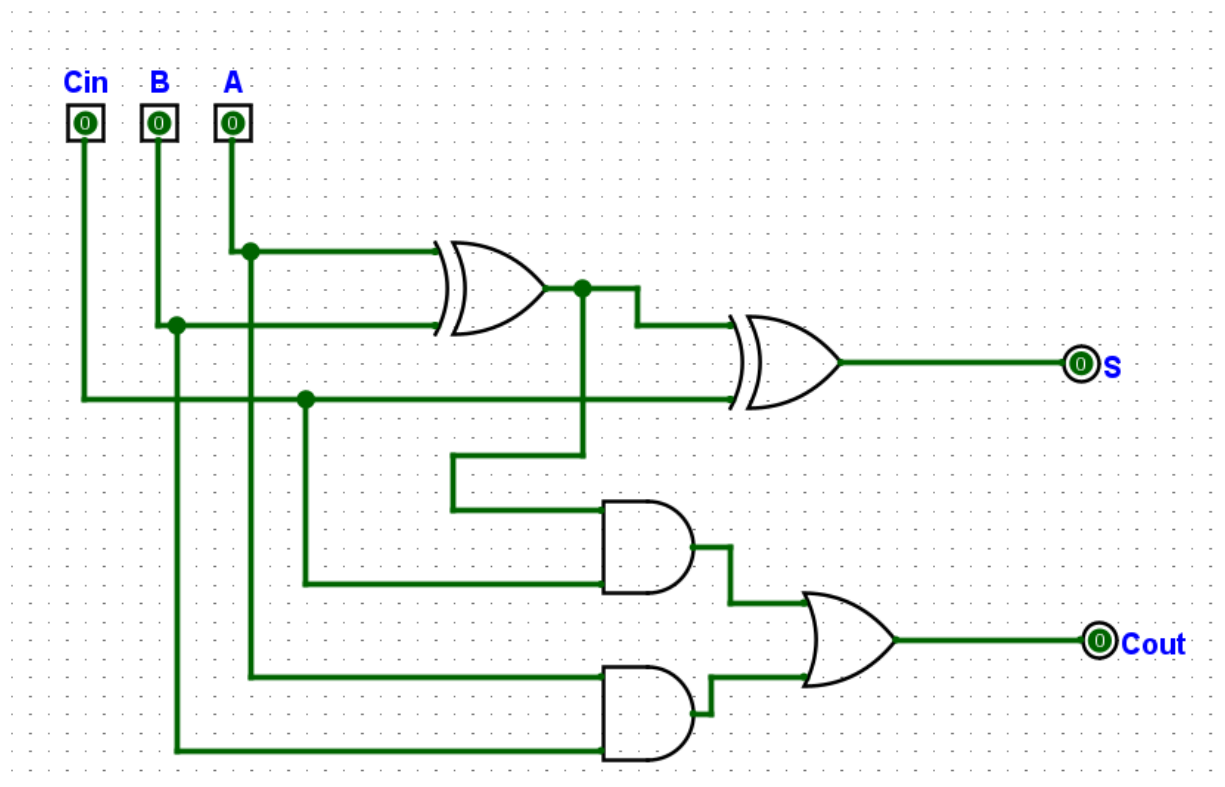
- **Hardware Implementation:** The circuit uses specialized logic to handle both signed and unsigned comparisons:
 - **Signed Comparison:** An XOR gate compares the Sign (the most significant bit of the result) and the Overflow flag. If the sign and overflow bits are different (Sign XOR Overflow = 1), the condition $A < B$ is true.
 - **Unsigned Comparison:** The logic monitors the Carry-Out signal. In unsigned subtraction, a Carry-Out of 0 indicates that a borrow occurred, meaning $A < B$. This signal is passed through a NOT gate to generate the boolean result.
- **Result Selection and Extension:** The isUnsigned control signal drives a Multiplexer to choose between the signed (XOR) or unsigned (NOT) logic path. The resulting 1-bit value is then passed through a zero-extender to output a full 32-bit word (0x00000001 for true or 0x00000000 for false).



2.3 Arithmetic Unit

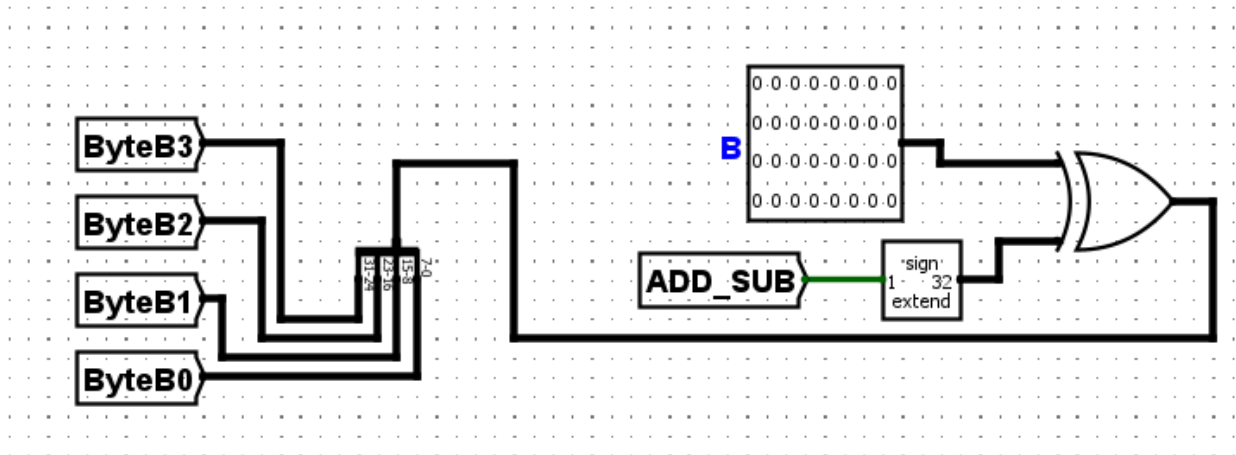
The Arithmetic Unit is the core computational block responsible for performing 32-bit signed addition and subtraction.

- Ripple Carry Adder (RCA):** The unit is built using 32 full adder circuits chained together in a ripple carry configuration. Each full adder processes one bit of A and B along with a carry-in (Cin) to produce a sum bit (S) and a carry-out (Cout).



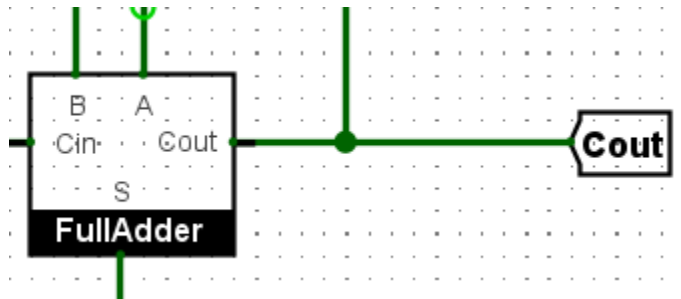
- Adder/Subtractor Logic:** The unit uses a unified design to handle both operations. To perform subtraction ($A - B$), the unit implements the 2's complement identity $A + B' + 1$.

This is achieved by passing each bit of operand B through an XOR gate controlled by the SUB signal; when SUB is high, operand B is bitwise inverted, and the Carry-In of the first bit is set to 1.

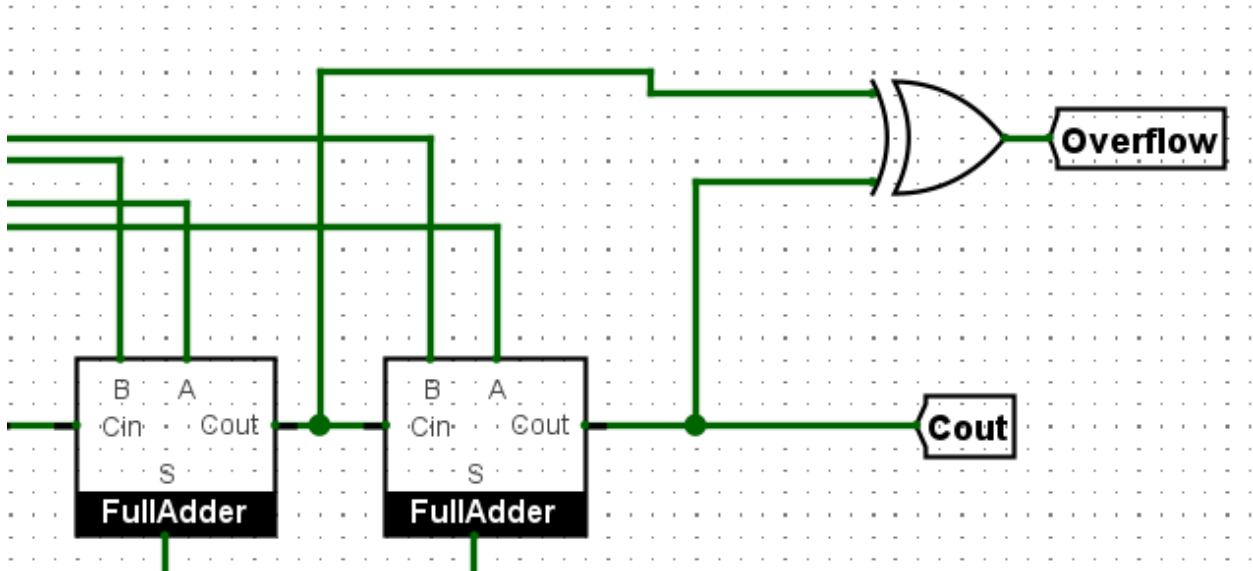


- **Flag Generation:**

- **Carry-Out:** The final carry-out from the 31st bit represents the unsigned carry (or borrow in subtraction).

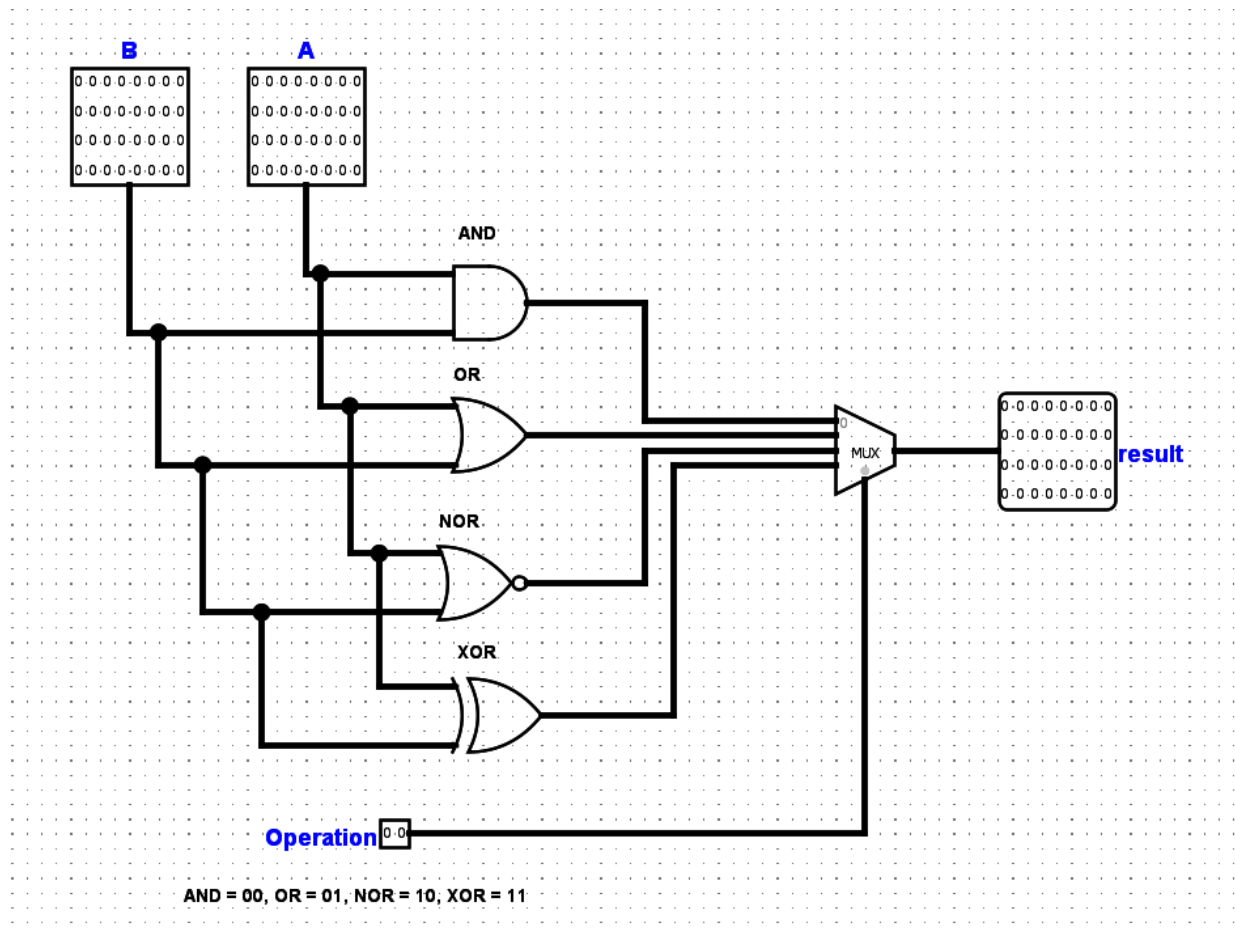


- **Overflow Detection:** Signed overflow is detected by XORing the carry-in and carry-out of the most significant bit (bits 30 and 31). If the carries differ, a signed overflow has occurred, indicating the result has exceeded the representable 32-bit range.



2.4 Logical Operation Unit

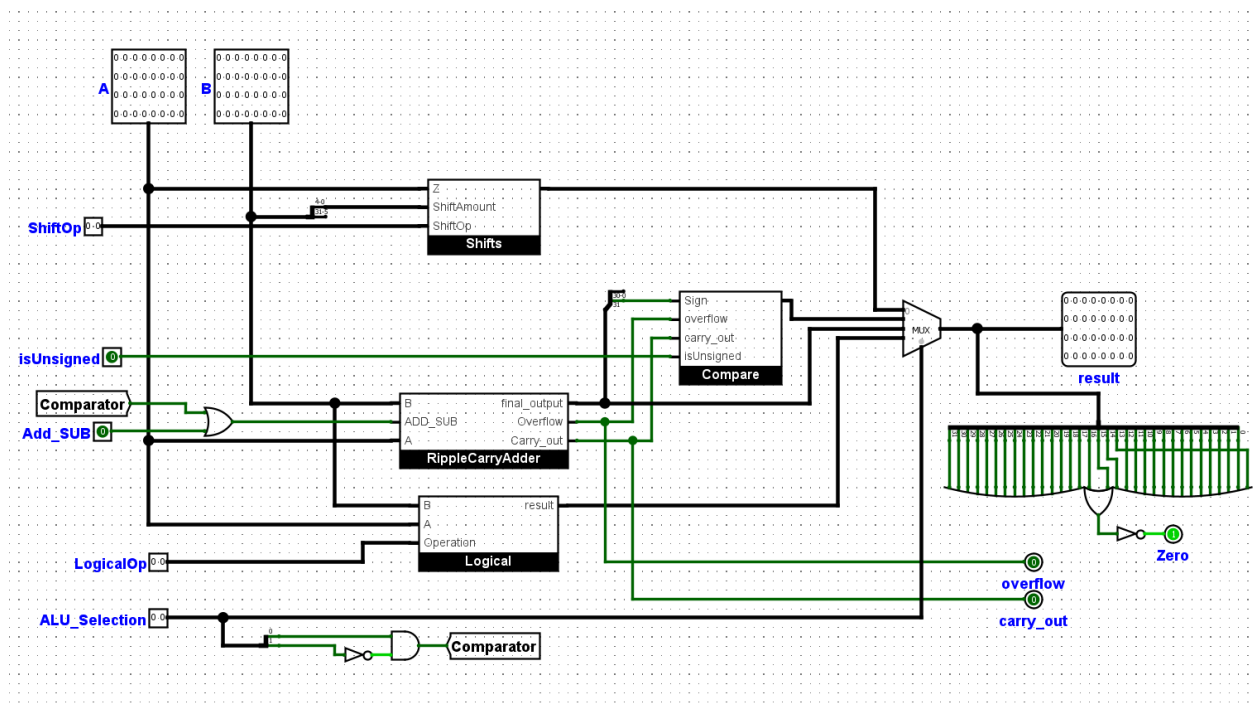
This unit performs bitwise AND, OR, NOR, and XOR operations using a dedicated 32-bit 4-to-1 multiplexer controlled by the lower 2 bits of the LogicalOp signal.



2.5 Main ALU Integration and Control Logic

The top-level circuit coordinates the data flow between functional units and handles specialized flag detection.

- Operand and Shift Routing:** Operand A is distributed to all units. For the Shifter, a splitter extracts the lower 5 bits of operand B (bits 4 to 0) to provide the ShiftAmount.
- Integrated Subtraction Control:** Subtraction is triggered through two stages of logic. First, an AND gate is utilized to decode the ALU_Selection bits (detecting the binary '01' state) to identify when a Comparison operation is active. Second, an OR gate combines this decoded signal with the manual Add_SUB input. This forces the adder into subtraction mode whenever the Add_SUB signal is high or when the Comparator logic is active, ensuring comparisons are performed correctly via subtraction.
- Sign Bit Extraction:** The sign bit required for signed comparisons is not a separate pin but is extracted directly from the most significant bit (bit 31) of the RippleCarryAdder result bus using a splitter.
- Zero Flag Logic:** Since Logisim-evolution does not feature a native 32-input NOR gate, the Zero flag is implemented using a 32-bit wide OR gate connected to the final result bus, followed by an inverter. This ensures the Zero flag is high only when all 32 output bits are low.
- Final Multiplexing:** A 4-to-1 32-bit Multiplexer uses the ALU_Selection signal to choose between the outputs of the Shifter, Comparator, Adder, and Logical units.



3. Functional Simulation

3.1 Test Table

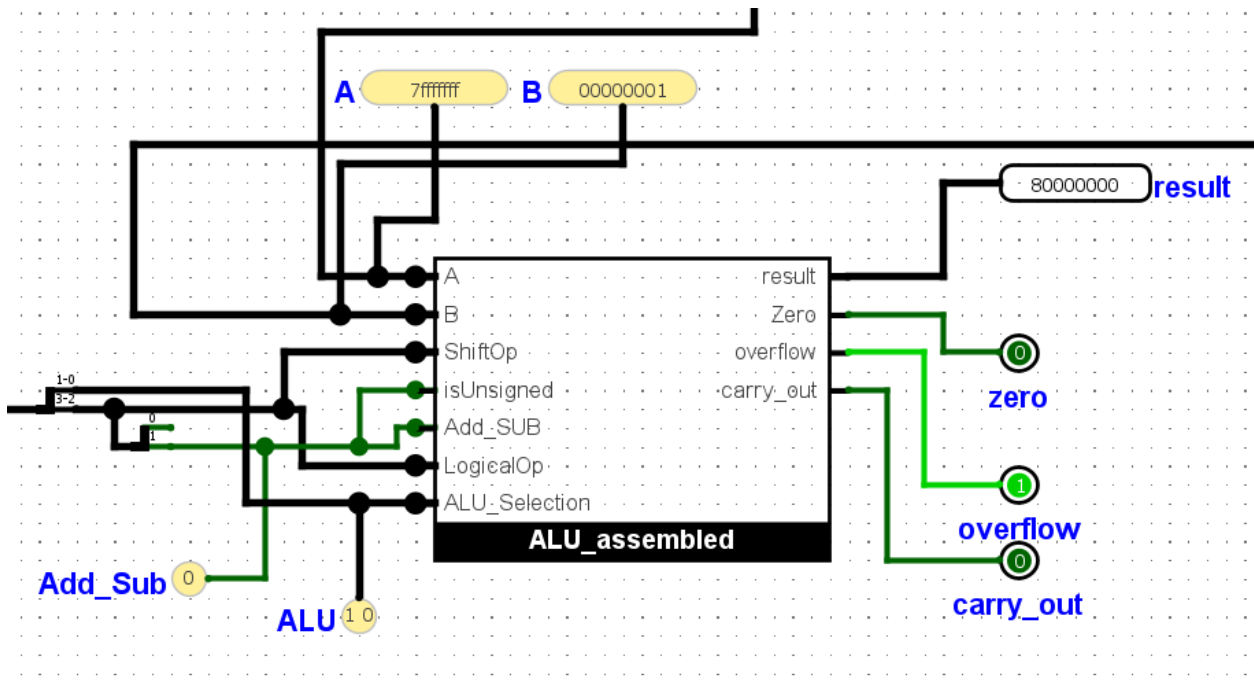
The table below represents the cycle by cycle behavior of the ALU. The Overflow and CarryOut flags may assert during logical or shift operations because the Arithmetic Unit continues to process the inputs in the background even though the final Multiplexer does not select its output for the result bus.

Cycle	ALUCtrl	A (hex)	B (hex)	Result (hex)	Zero	Overflow	CarryOut
0	ROR [0]	AAAAAAAA5	00000004	5AAAAAAAA	0	0	0
1	SLL [4]	AAAAAAAA5	00000004	AAAAAA50	0	0	0
2	SRL [8]	AAAAAAAA5	00000004	0AAAAAAAA	0	0	1
3	SRA [C]	AAAAAAAA5	00000004	FAAAAAAA	0	0	1
4	SLT [1]	00000002	00000005	00000001	0	0	0
5	ADD [2]	7FFFFFFF	00000001	80000000	0	1	0
6	ADD [2]	00000005	00000003	00000008	0	0	0
7	SUB [6]	80000000	80000000	00000000	1	0	1
8	AND [3]	F0F0F0F0	0F0F0F0F	00000000	1	0	0
9	OR [7]	F0F0F0F0	0F0F0F0F	FFFFFFFF	0	0	0
10	NOR [B]	00000000	00000000	FFFFFFFF	0	1	0
11	XOR [F]	AAAAAAAA	55555555	FFFFFFFF	0	1	1

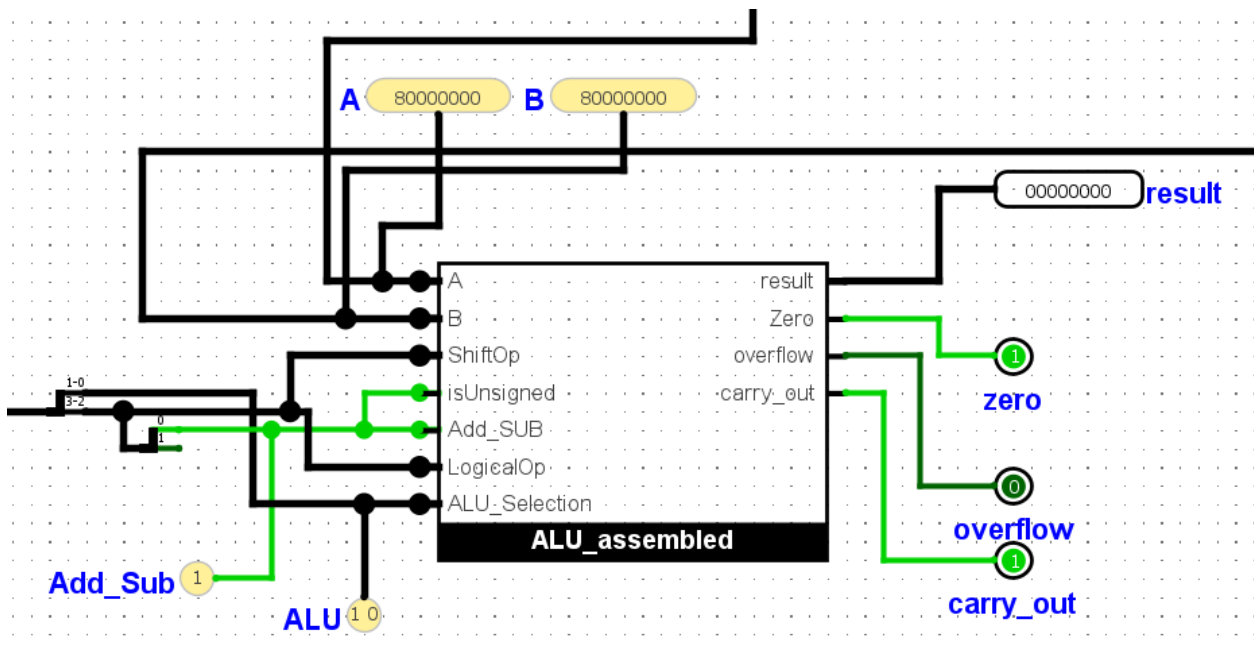
Note: Individual functional test screenshots for all 12 cycles shown in the table above are included in the submitted project ZIP file.

3.2 Key Verification Snapshots

- The image below shows that adding 0x7FFFFFFF and 0x1 results in 0x80000000 with the Overflow flag asserted.



- Subtracting equal values results in 0x00000000, correctly asserting the Zero flag via the NOR-tree detector.

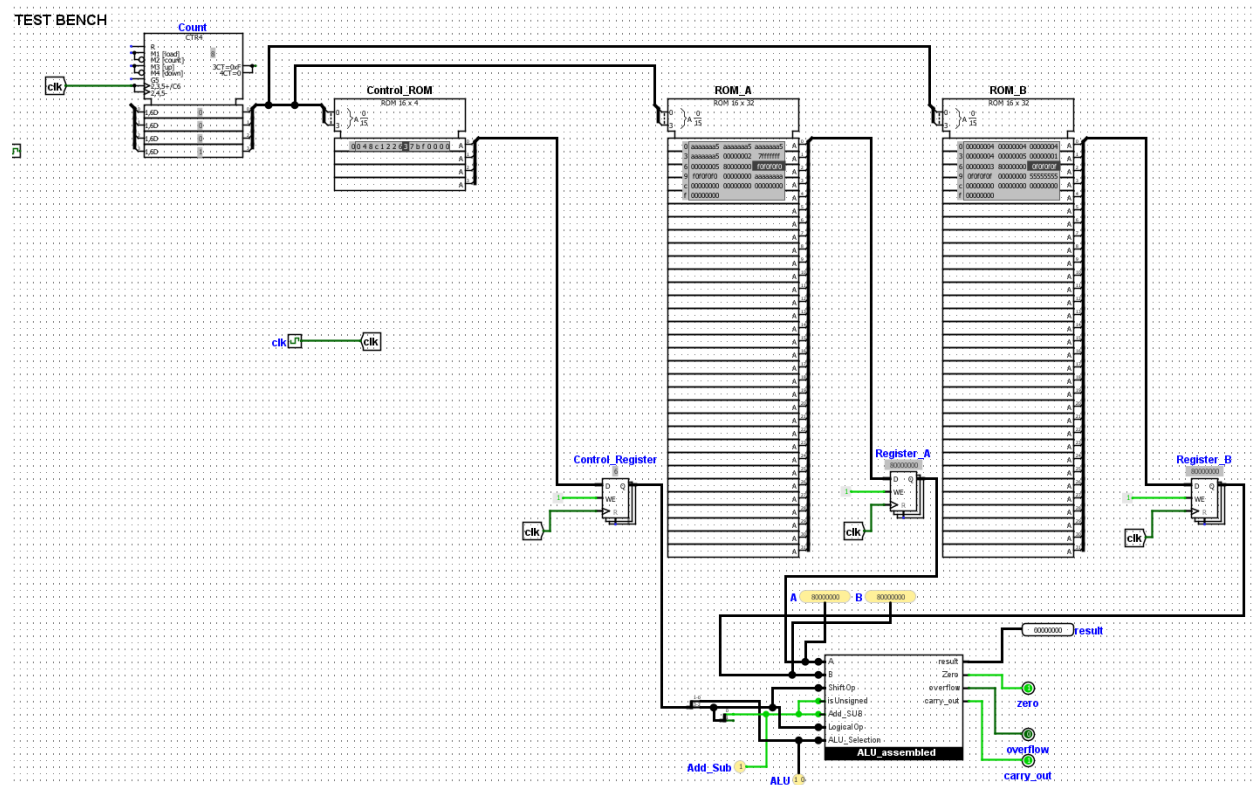


4. Chronogram Timing Analysis

Verification was automated using a clock-driven testbench.

4.1 Testbench Construction

To automate the verification process, a clocked testbench was constructed using a triple ROM architecture. Since Logisim-evolution restricts ROM components to a maximum 32-bit data width, three separate ROMs were banked together to accommodate the full 68-bit test vector. One 4-bit ROM stores the ALUctrl hex codes, while two 32-bit ROMs store operands A and B. All three ROMs are addressed by a single 4-bit counter, ensuring synchronized delivery of test data to the ALU input registers on every clock cycle.



4.2 Waveform Observation

[illegible]

4.3 Interpretation of Results

The following two cycles demonstrate the ALU's ability to handle boundary conditions and flag assertions during real-time operation.

Cycle 5: Signed Overflow Detection

In this cycle, the ALU performs a signed addition that exceeds the capacity of a 32-bit signed integer.

- Input State: Operand A is set to 0x7FFFFFFF (the maximum positive signed 32-bit integer) and B is 0x00000001.
- Control Signal: The ALU control bus carries the value 2 (ADD).
- Observation: On the rising edge of the clk, the result transitions to 0x80000000.
- Correctness: The overflow flag asserts high. This is correct because the sum of two positive numbers produced a negative bit pattern (indicated by the MSB being 1), which triggers the overflow detection logic (XOR of carry-ins to bits 30 and 31).

Cycle 7: Zero Flag and Subtraction Logic

This cycle verifies the Subtractor and the Zero-detection circuitry.

- Input State: Both operands A and B are set to 0x80000000.
- Control Signal: The ALU control bus carries the value 6 (SUB).
- Observation: The result bus transitions to 0x00000000.
- Correctness:
 - Zero Flag: The zero flag asserts high, confirming the 32-bit NOR-tree detector correctly identified an all-zero output.
 - CarryOut: The carry_out flag asserts high. In the ALU's subtraction logic ($A + B' + 1$), a carry-out of 1 signifies that no borrow was needed.

5. Specification to Testable Design Implementation

Refining the RISC-V ALU specification into a testable hardware design required bridging the gap between static logic and temporal behavior. While the ISA defines functional mappings, the implementation required a Test Harness to ensure these mappings remain stable across clock cycles.

To achieve this, I implemented a ROM-based test vector system. Because Logisim-evolution restricts ROM data width to 32 bits, I designed a parallel-synchronous ROM architecture to deliver the full 68-bit test vector (Control + A + B). This modularity allowed for rapid debugging

of edge cases like Overflow and Zero detection that would be difficult to verify through manual input toggling alone.